

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №4

Тема: Основи проектування

Виконав:
студент групи ІА-34
Єресько І.С
Перевірив:
Мягкий М.Ю

Київ-2025

Короткі теоретичні відомості

Поняття шаблону проектування

Будь-який шаблон (патерн) проектування, що використовується під час створення інформаційних систем, є формалізованим описом типового завдання проектування, перевіреним ефективним способом його розв'язання та рекомендаціями щодо застосування цього підходу в різних умовах. Крім цього, кожен патерн має усталену назву, яка загальноприйнята у професійному середовищі.

Коректно сформульований шаблон дає можливість, одного разу знайшовши оптимальне рішення, багаторазово його повторно використовувати. Водночас ключовим початковим етапом роботи з патернами є якісне моделювання предметної області. Це необхідно як для чіткого формулювання задачі, так і для правильного добору відповідних шаблонів проектування.

4.2.2. Шаблон «Singleton»

Призначення:

Шаблон «Singleton» (Одинак) - це підхід, за якого в об'єктно-орієнтованій системі певний клас може мати лише один екземпляр. За потреби можна обмежити кількість створюваних об'єктів фіксованим числом n . Зазвичай цей єдиний об'єкт зберігається як статичне поле самого класу.

Проблема:

Одинак доцільно використовувати в ситуаціях, коли:

- фізично може існувати тільки певна кількість екземплярів класу;
- необхідно централізовано контролювати всі операції, що проходять через конкретний клас.

Таким чином, «Singleton» фактично вирішує дві задачі, але одночасно й порушує принцип єдиної відповідальності.

Рішення:

Уявімо поширений випадок: будь-яка програма має файл конфігурації (.ini, .xml тощо). Такий файл єдиний, і тому створювати багато об'єктів для роботи з ним недоцільно - тут ідеально підходить шаблон «Одинак».

Інший приклад - система з двома підсистемами, між якими працює лише один канал зв'язку. Через обмеження: у певний момент можна надсилати дані тільки в одному напрямку, - Singleton виступає єдиним об'єктом сеансу і водночас контролює логіку перевірок, чергу запитів, можливість відправлення тощо.

Переваги та недоліки:

Хоч «Singleton» і забезпечує існування одного екземпляра та глобальну точку доступу, сьогодні його часто вважають анти-патерном. Причина - він фактично створює глобальні змінні зі станом, які складно тестувати, важко відслідковувати та підтримувати. Зміна такої архітектури призводить до масштабних правок коду.

Альтернативою можуть бути статичні змінні, блокування, замикання та інші механізми контролю доступу.

- + гарантує єдиний екземпляр класу
- + надає глобальну точку доступу
- порушує принцип єдиної відповідальності
- приховує недоліки архітектури

4.2.3. Шаблон «Iterator»

Призначення:

Шаблон «Iterator» (Ітератор) описує спосіб доступу до елементів колекції без необхідності знати, як саме вона реалізована всередині.

Колекція відповідає за зберігання даних, а ітератор - за послідовний обхід.

Ітератори можуть мати різні алгоритми обходу: зліва направо, справа наліво, за парними позиціями тощо - і все це незалежно від того, чи це масив, список, дерево або граф.

Проблема:

Більшість колекцій здаються простими, але існують складні структури на зразок дерев чи графів.

Користувачеві потрібно мати можливість пройти колекцію послідовно, але робити це різними способами. Якщо додавати логіку всіх можливих обходів у саму колекцію - вона втрачає свою основну роль: ефективне зберігання даних.

Рішення:

Обхід колекції винесли в окремий клас - ітератор.

Об'єкт ітератора знає: поточну позицію, правила переходу, коли обхід завершено.

Колекція не має інформації про кількість ітераторів, які її обходять. Щоб додати новий тип обходу - потрібно створити новий клас ітератора, не змінюючи саму колекцію.

Типові методи ітератора:

First() - перейти до першого елемента;

Next() - перейти до наступного;

IsDone - ознака завершення;

CurrentItem - отримати поточний елемент.

Переваги та недоліки:

Ітератори дозволяють уніфікувати обхід колекцій незалежно від того, як ті влаштовані.

+ підтримує різні способи обходу

+ спрощує структуру класів колекцій

– зайвий у простих випадках, де вистачає звичайного циклу

4.2.4. Шаблон «Proxy»

Призначення:

«Proxy» (Проксі) - це об'єкт-посередник, який замінює реальний об'єкт і бере на себе частину його функцій або спрощує взаємодію. Наприклад: браузері раніше показували «заглушки» для зображень, поки ті завантажувалися.

Проблема:

Система активно використовує зовнішній сервіс підписання документів (наприклад, DocuSign). Зі збільшенням кількості користувачів різко виросла кількість запитів, що призвело до високих витрат.

Після аналізу стало зрозуміло, що підписані документи можна групувати: достатньо відправляти їх раз на годину, а не при кожному клієнтському запиті.

Рішення:

Створюється проксі-клас, який реалізує той самий інтерфейс **IDocSignManager**, але працює як буфер:

- накопичує запити та відправляє їх пакетами;
- раз на годину отримує підписані документи;
- а клієнтам одразу повертає дані з локальної бази.

Основний DocSignManager продовжує працювати з сервісом, але значно рідше.

У результаті використання зменшилось приблизно на 40%, і основні витрати тепер залежать не від кількості запитів, а від розміру файлів.

Переваги та недоліки:

- + можна додати новий проміжний рівень без змін у клієнтському коді
- + з'являється можливість контролювати життєвий цикл об'єктів
- імовірно зниження швидкості роботи
- ризик отримати некоректні або несвоєчасні відповіді

4.2.5. Шаблон «State»

Призначення:

Шаблон «State» (Стан) дозволяє змінювати поведінку об'єкта залежно від його внутрішнього стану.

Приклади:

тариф банківської картки впливає на відсотки;

тарифний план хостингу визначає обсяг доступних послуг.

Усі залежні від стану змінні й методи виділяються в інтерфейс **State**, а кожен стан реалізується у власному класі (**ConcreteStateA**, **ConcreteStateB**). Зміна об'єкта стану в контексті миттєво змінює поведінку всієї системи.

Проблема:

У системі з мобільними клієнтами та центральним сервером необхідно, щоб **Listener**: не приймав запити під час запуску сервера, працював повноцінно під час нормальної роботи, завершував роботу коректно після команди **shutdown**.

Це три різні стани з різною логікою.

Рішення:

Створюється інтерфейс **IListenerState** і 3 реалізації: **InitializingState**, **OpenState**, **ClosingState**.

Listener делегує всі виклики об'єктові стану.

Переходи:

при запуску - InitializingState (ігнорує запити)

після готовності - OpenState (повна робота)

при вимкненні - ClosingState (не приймає нових запитів, але відправляє відповіді)

Переваги та недоліки:

- + логіка кожного стану винесена в окремий клас
- + стани можна використовувати в різних контекстах
- + код контексту стає чистішим
- + легко додавати нові стани
- контекст ускладнюється механізмом перемикання станів

4.2.6. Шаблон «Strategy»

Призначення:

Шаблон «Strategy» (Стратегія) дає змогу замінювати алгоритм роботи об'єкта іншим алгоритмом, який досягає тієї ж мети, але іншим способом. Класичний приклад - різні алгоритми сортування, кожен зі своєю реалізацією.

«Strategy» зручна там, де існують різні «політики» обробки даних.

За логікою схожа на «State», але використовується для інших цілей:

стани змінюються залежно від об'єкта, а стратегії - для вибору поведінки, яка не залежить від стану контексту.

Тема: Вступ до паттернів проектування.

Мета: Вивчити структуру шаблонів «Singleton», «Iterator», «Proxy», «State», «Strategy» та навчитися застосовувати їх в реалізації програмної системи.

Хід роботи

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру

вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або

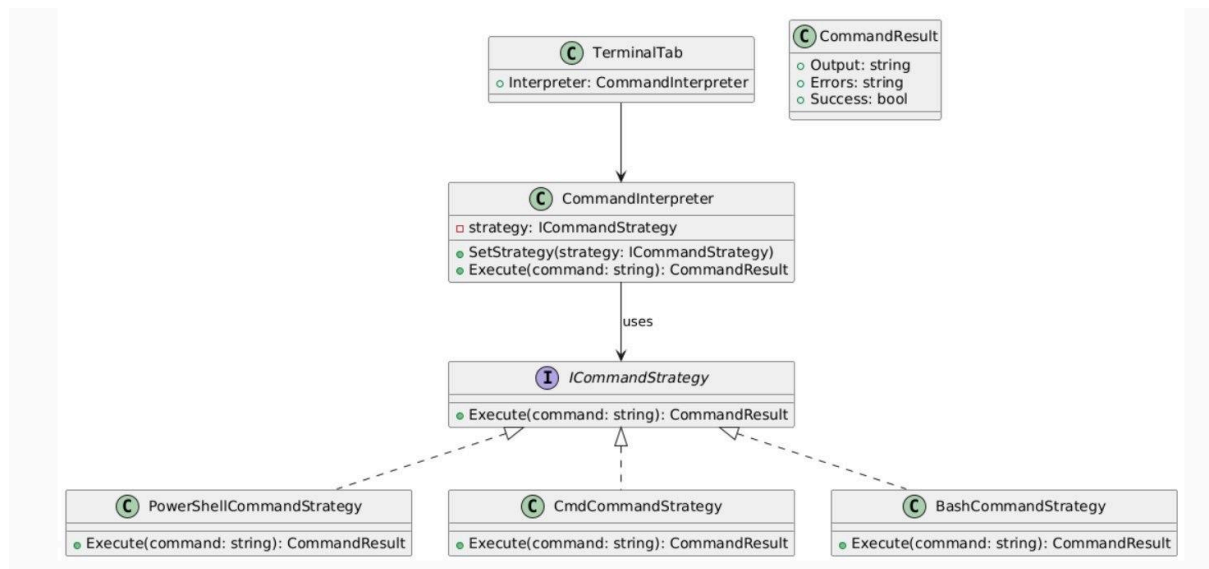


Рис 1 :Strategy class diagram

Висновок

У ході виконання роботи було розглянуто та проаналізовано основні шаблони проєктування: **Singleton**, **Iterator**, **Proxy**, **State** та **Strategy**. Вивчено їх призначення, структуру, переваги використання та типові сфери застосування. Застосування цих патернів дозволяє підвищити гнучкість, масштабованість та підтримуваність програмних систем.

Під час виконання завдання було побудовано UML-структури **Strategy** шаблону. Таким чином, опанування цих шаблонів є важливим етапом у формуванні навичок проєктування якісного програмного забезпечення, що відповідає принципам SOLID та кращим практикам архітектури.

Контрольні запитання

1. Що таке шаблон проєктування?

Шаблон проєктування - це типове, багаторазове рішення для поширеної архітектурної або програмної проблеми. Він не є готовим кодом, а радше структурою, ідеєю чи підходом, яка допомагає будувати гнучкі та підтримувані програми. Шаблон задає форму взаємодії між класами та об'єктами, дозволяючи мінімізувати дублювання, підвищити читабельність і полегшити розширення системи.

2. Навіщо використовувати шаблони проєктування?

Шаблони використовують для того, щоб уникати типових помилок, покращити архітектурну якість коду та спростити подальше супроводження системи. Вони дозволяють програмісту описувати рішення на високому рівні, не винаходячи велосипед для поширених задач, а також забезпечують спільну "мову" між розробниками, які швидше розуміють структуру та логіку чужого коду.

3. Яке призначення шаблону «Стратегія»?

«Стратегія» дозволяє інкапсулювати різні алгоритми в окремі класи й динамічно перемикатися між ними під час роботи програми. Головна ідея полягає в тому, що об'єкт не містить логіку алгоритму всередині себе, а делегує її окремій стратегії, завдяки чому код стає більш гнучким, легко розширюваним і позбавленим великих умовних конструкцій.

4. Структура шаблону «Стратегія».

Структура складається з абстрактного інтерфейсу стратегії, конкретних класів стратегій і контексту, який містить посилання на вибрану стратегію. Контекст викликає метод стратегії, не знаючи деталей реалізації, що дозволяє в будь-який момент замінити одну стратегію іншою.

5. Які класи входять у «Стратегію» та яка їх взаємодія?

До «Стратегії» входять інтерфейс Strategy, конкретні стратегії ConcreteStrategyA/B та клас Context. Інтерфейс визначає спільний метод, конкретні стратегії реалізують різні алгоритми, а Context містить

посилання на стратегію та викликає її метод. Контекст працює тільки з інтерфейсом, тому легко підмінювати стратегію під час виконання програми.

6. Яке призначення шаблону «Стан»?

Шаблон «Стан» дозволяє об'єкту змінювати свою поведінку залежно від внутрішнього стану. З точки зору клієнта здається, що об'єкт динамічно змінює свій клас. Це дозволяє уникати величезних switch-case конструкцій та підвищує логічну чистоту коду, бо кожен стан має власні правила поведінки.

7. Структура шаблону «Стан».

Зазвичай структура містить інтерфейс State, конкретні класи станів та контекст, що зберігає поточний стан. Контекст делегує виконання дій стану, а сам стан може ініціювати зміну стану на інший, передаючи контексту новий об'єкт стану.

8. Які класи входять у «Стан» та яка їх взаємодія?

У шаблон входять інтерфейс State, конкретні стани ConcreteStateA/B і Context. Контекст володіє поточним станом і викликає його методи, а конкретний стан визначає, як саме поводитися в цій ситуації. Стани можуть змінювати контекст так, що той починає використовувати інший стан, завдяки чому поведінка програми змінюється природно та структуровано.

9. Яке призначення шаблону «Ітератор»?

Ітератор дає змогу послідовно обходити елементи колекції, не оголюючи її внутрішню структуру. Завдяки цьому різні типи колекцій можна обходити однаковим способом, що спрощує роботу з даними та дозволяє безпечніше взаємодіяти зі структурами даних.

10. Структура шаблону «Ітератор».

Структура складається з інтерфейсу Iterator, який визначає методи переходу між елементами, конкретного ітератора, що знає, як обходити

певну колекцію, та інтерфейсу Aggregate (або Collection), який створює ітератор для своєї внутрішньої структури.

11. Які класи входять у «Ітератор» та яка між ними взаємодія?

До шаблону входять інтерфейс Iterator, конкретна реалізація ConcreteIterator, інтерфейс Aggregate і конкретна колекція ConcreteAggregate. Колекція створює конкретний ітератор, який містить інформацію про поточну позицію та методи next(), hasNext() тощо. Ітератор взаємодіє з колекцією через її публічні методи, не маючи доступу до внутрішньої реалізації.

12. В чому полягає ідея шаблону «Одинак»?

«Одинак» (Singleton) гарантує, що клас матиме лише один екземпляр у системі та надає глобальний доступ до нього. Основна ідея - централізований контроль ресурсу або сервісу, який не повинен існувати в кількох копіях, наприклад, єдиний логер або керівник підключення до БД.

13. Чому «Одинак» вважають анти-шаблоном?

Singleton критикують тому, що він створює приховану глобальну залежність, ускладнює тестування та порушує принципи чистої архітектури. Програма, що сильно залежить від одинаків, стає менш гнучкою, трудно адаптується для юніт-тестів і вимагає занадто багато прихованого стану, який важко контролювати.

14. Яке призначення шаблону «Проксі»?

Проксі створює замісник або сурогат для іншого об'єкта, щоб контролювати доступ до нього. Він дозволяє додати додаткову поведінку - кешування, ліниве створення, контроль безпеки, логування - без зміни оригінального класу. Проксі виглядає і поводить себе як реальний об'єкт, але виконує додаткову роботу перед або після звернення.

15. Структура шаблону «Проксі».

Структура включає інтерфейс Subject, який визначає поведінку справжнього об'єкта, клас RealSubject з реальною логікою та клас Proxy, що також реалізує Subject і містить посилання на RealSubject. Проксі перехоплює виклики й вирішує, коли саме передати їх реальному об'єкту.

16. Які класи входять у «Проксі» та яка між ними взаємодія?

До шаблону входять Subject, RealSubject і Proxy. Клієнт працює з інтерфейсом Subject, не знаючи, чи звертається він до реального об'єкта чи до проксі. Proxy отримує виклики від клієнта, виконує додаткову логіку та за потреби делегує роботу RealSubject. Це дозволяє прозоро керувати